Lecture # 15

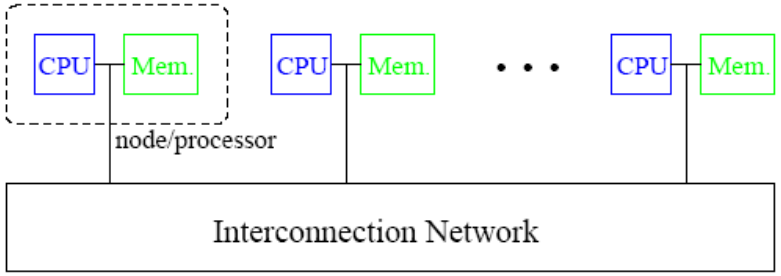
By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Introduction to Message Passing Systems

Message passing systems allow processes running on different machines or different nodes within a distributed memory system to communicate with each other by sending and receiving messages.

- **Communication in Distributed Memory**: In distributed memory systems, each process has its own private memory. Processes must communicate explicitly through message passing since they cannot access each other's memory directly.
- **Benefits**: This approach is efficient for parallel programming as it provides fine-grained control over data distribution and communication, enabling optimization of performance.



Overview of MPI and MPICH Libraries

- **MPI (Message Passing Interface)**:
  - **Standard**: MPI is an industry-standard interface for message passing, defined by the MPI Forum. It provides a common way to write parallel programs that can run on various platforms.
  - **Features**: It includes features like point-to-point communication, collective communication, synchronization, and data management.
- **MPICH**:
  - **Implementation**: MPICH is one of the most popular and portable implementations of the MPI standard.
  - **Cross-Platform**: It works on various platforms and is designed to work with different architectures and interconnects.
  - **Performance**: MPICH is known for its high performance and ability to run on a wide range of systems, from small clusters to large supercomputers.

MPI and MPICH Applications

- **Scientific Simulations**:
  - MPI and MPICH are widely used in scientific computing for simulations in physics, chemistry, biology, and other domains.
- **Data Processing**:
  - They facilitate large-scale data processing tasks, such as data analysis and big data applications.
- **Distributed Tasks**:
  - Commonly used for tasks like parallel computing, distributed database management, and cloud computing.

Different implementations

MPICH

- Open source
- CH stands for Chamaleon portability system
- Built in collaboration with UBC, Microsoft, IBM and Cray

OpenMPI Implementation

- Open source (Git)
- Collaborators from both academia and industry

Basic concepts of MPI

Communication Domain

- Processes that can communicate with each other
- Stored in **communicator**
- Communicator type: MPI_Comm
- Predefined default: MPI COMM WORLD

MPI Functions

```
/* Set up */
int MPI_Init(int *argc, char ***argv);

/*Tear down */
int MPI_Finalize()

/*Total processes */
int MPI_Comm_size(MPI_Comm comm, int *size);

/*Local process index */
int MPI_Comm_rank(MPI_Comm comm, int *rank);
```

Typical MPI Initialization

```
int main(int argc, char **argv)
{
    int num_procs;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("%d: hello (p=%d)", rank, num_procs);

    /* Do many things, all at once */


    MPI_Finalize();
}
```

Primitive Communication

```
// Send
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)

//Receive
int MPI_Recv(void *buf, int count,
               MPI_Datatype datatype,
               int source, int tag,
               MPI Comm comm,
               MPI_Status *status)
```

MPI Data Types

MPI_CHAR

MPI_SIGNED_CHAR

MPI_BYTE

MPI_WCHAR

MPI_SHORT

MPI_UNSIGNED_SHORT

MPI_INT

MPI_UNSIGNED

MPI_LONG

MPI_FLOAT

MPI_DOUBLE

MPI_LONG_DOUBLE

MPI_LONG_LONG_INT

MPI_UNSIGNED_LONG_ LONG

MPI_LONG_LONG

MPI Status

```
typedef struct MPI_Status
{
    int MPI_SOURCE;
    int MPI_TAG;
    int MPI_ERROR;
}
```

Types of Point-to-Point Operations

MPI point-to-point operations typically involve message passing between two, and only two, different MPI tasks.

- One task is performing a send operation and the other task is performing a matching receive operation.

There are different types of send and receive routines used for different purposes.

- Any type of send routine can be paired with any type of receive routine.

Synchronous Send (MPI_Ssend)

- The sender waits until the matching receive is started on the receiving side. Ensures both sender and receiver are synchronized.

Types of Point-to-Point Operations

Blocking Send / Blocking Receive (MPI_Send, MPI_Recv)

- Blocking Send: The function returns only after the message data and envelope are safely stored (either sent or buffered).
- Blocking Receive: The function waits until the message has been fully received and stored in the buffer.

Non-blocking Send / Non-blocking Receive (MPI_Isend, MPI_Irecv)

- Starts the send/receive operation and returns immediately. The actual communication proceeds in the background.
- Requires MPI_Wait or MPI_Test to ensure completion.

Types of Point-to-Point Operations

Buffered Send (MPI_Bsend)

- Allows the program to send messages without waiting for a matching receive by using a user-provided buffer.
- Useful when you want to avoid blocking but can’t guarantee that a receive has been posted.

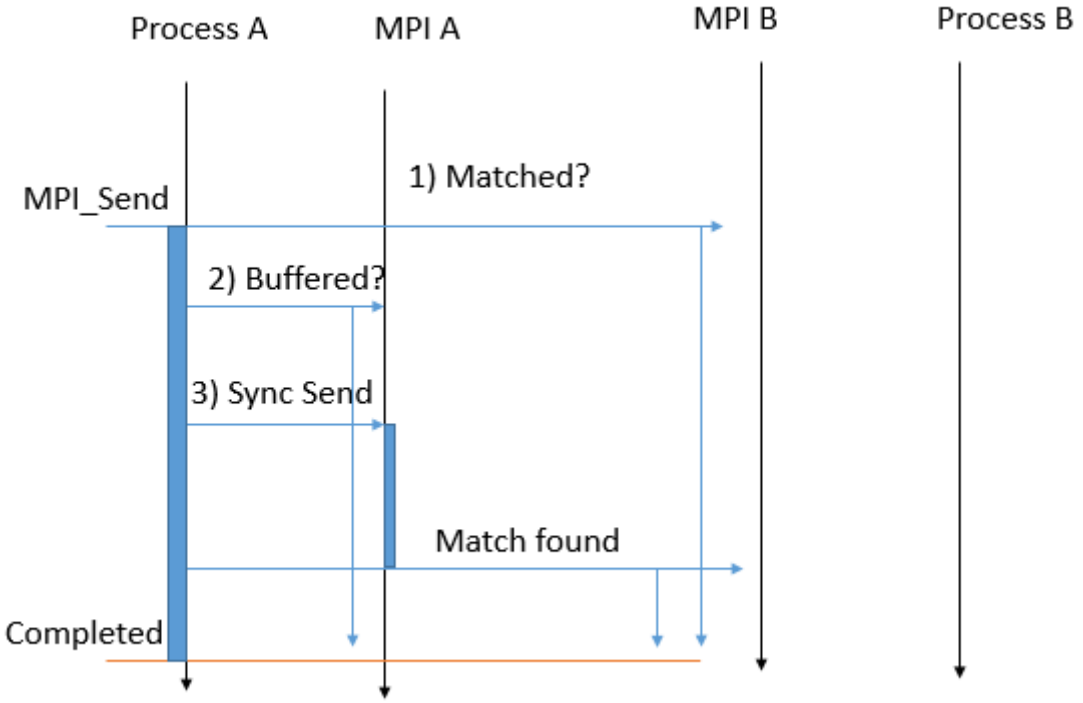
Combined Send/Receive (MPI_Sendrecv)

- Performs a send and a receive simultaneously.
- Useful for exchanging data between two processes without deadlock.

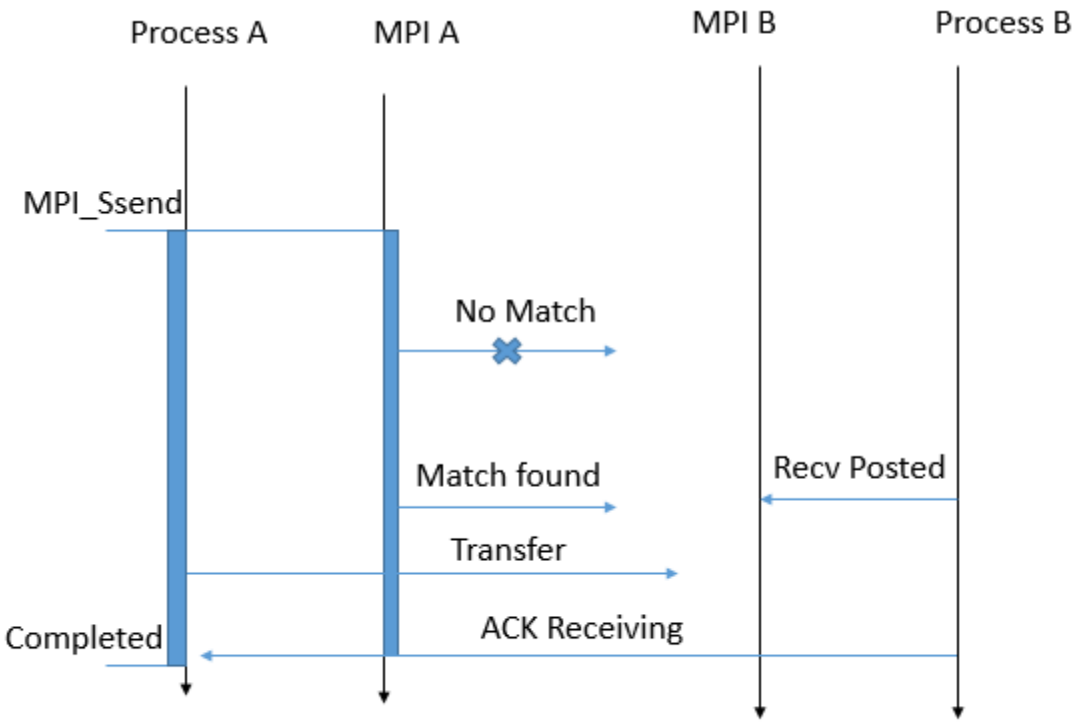
“Ready” Send (MPI_Rsend)

- Can only be used when the matching receive is guaranteed to be posted before the send. It's faster but unsafe if the receive is not already waiting.

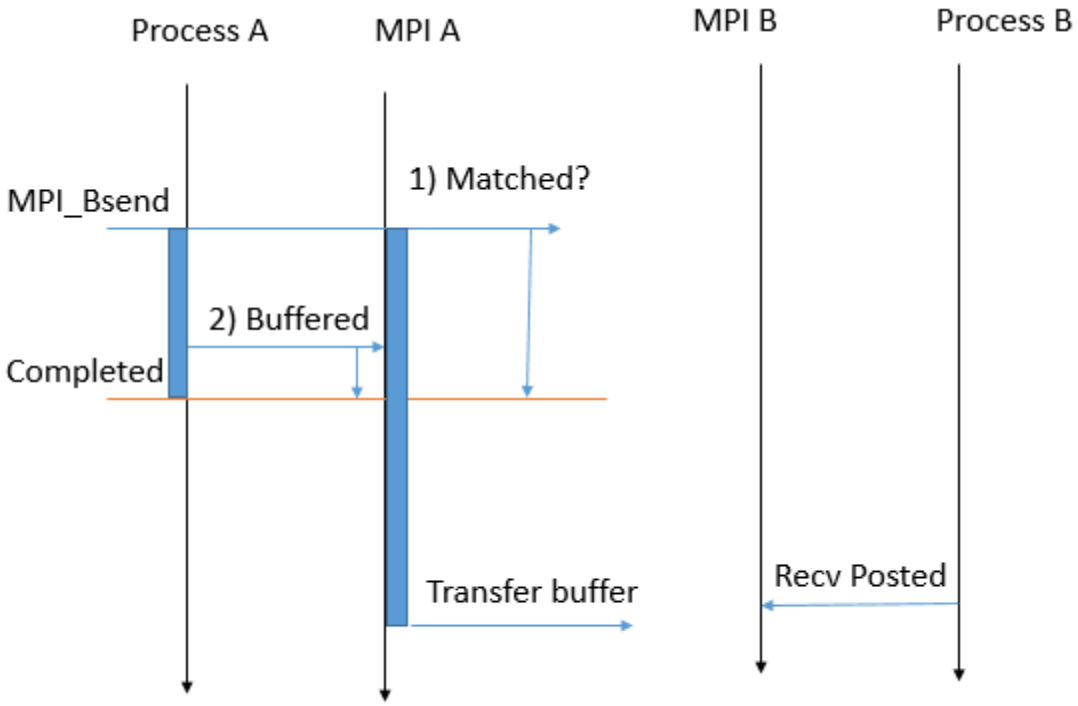
MPI_SEND


Process A
MPI A
MPI B
Process B
MPI_Send
1) Matched?
2) Buffered?
3) Sync Send
Match found
Completed

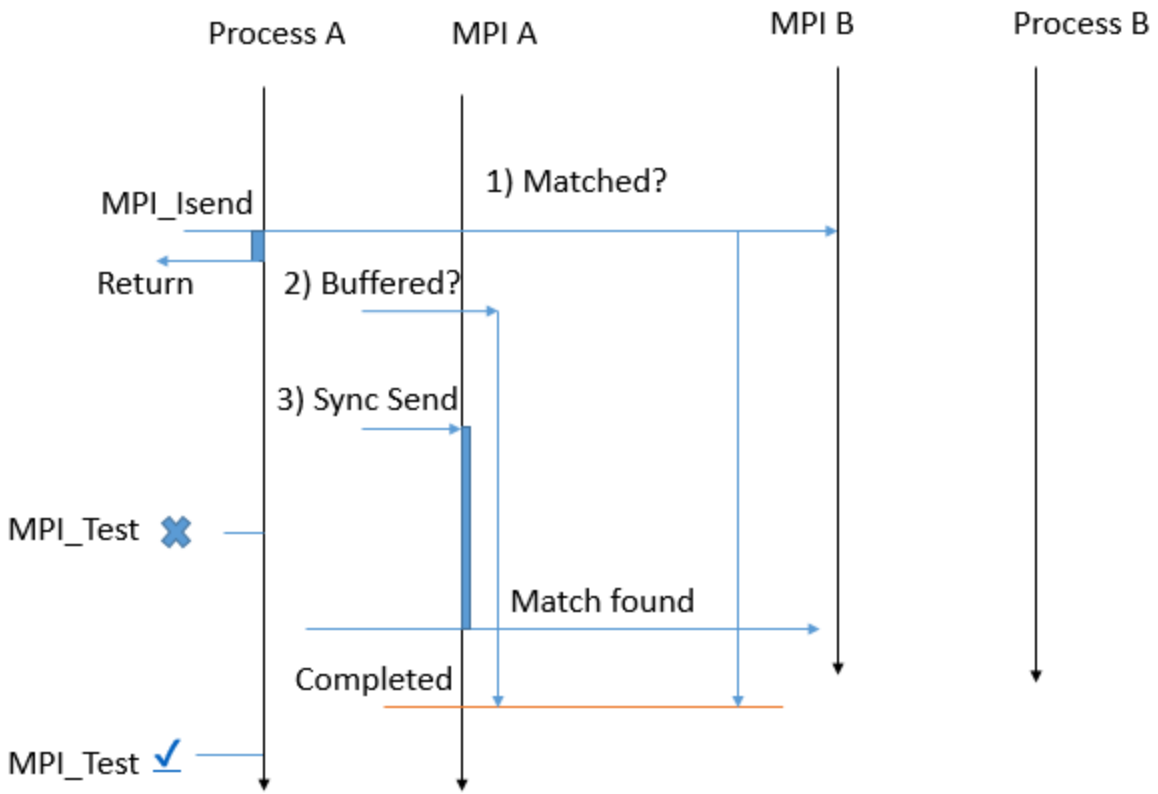
MPI_SSEND


Process A
MPI A
MPI B
Process B
MPI_Ssend
No Match
Match found
Recv Posted
Transfer
ACK Receiving
Completed

MPI_BSEND


Process A
MPI A
MPI B
Process B
MPI_Bsend
1) Matched?
2) Buffered
Completed
Transfer buffer
Recv Posted

MPI_ISEND


Process A
MPI A
MPI B
Process B
1) Matched?
MPI_Isend
Return
2) Buffered?
3) Sync Send
MPI_Test
Match found
Completed
MPI_Test

Buffering

In a perfect world, every send operation would be perfectly synchronized with its matching receive.

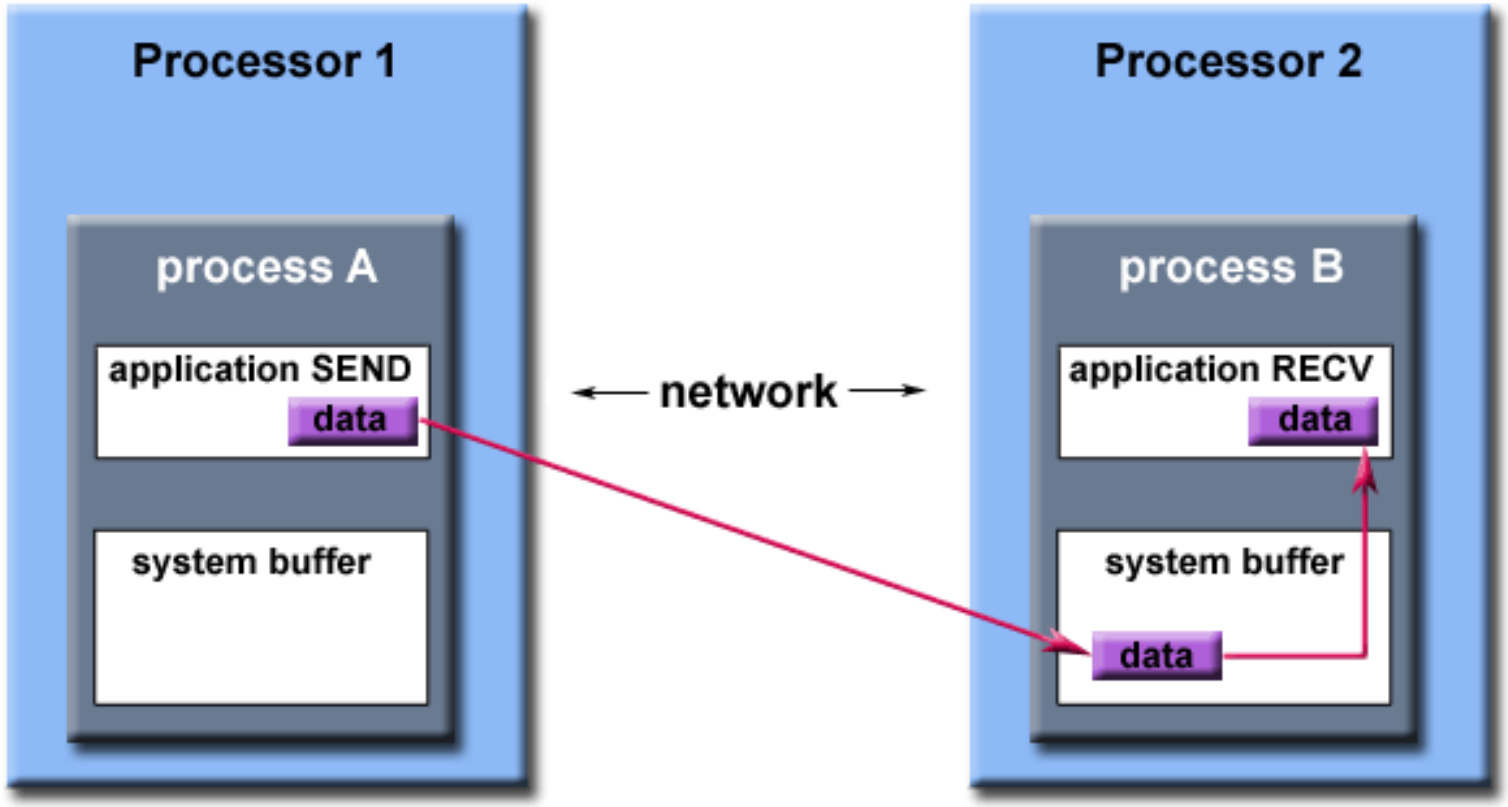
- This is rarely the case.

Somehow or other, the MPI implementation must be able to deal with storing data when the two tasks are out of sync.

Consider the following two cases:

- A send operation occurs 5 seconds before the receive is ready - where is the message while the receive is pending?
- Multiple sends arrive at the same receiving task which can only accept one send at a time - what happens to the messages that are “backing up”?

Buffering


Processor 1
process A
application SEND
data
system buffer
network
Processor 2
process B
application RECV
data
system buffer
data

Buffering

System buffer space is:

- Opaque to the programmer and managed entirely by the MPI library
- A finite resource that can be easy to exhaust
- Often mysterious and not well documented
- Able to exist on the sending side, the receiving side, or both
- Something that may improve program performance because it allows send - receive operations to be asynchronous.

User managed address space (i.e. your program variables) is called the application buffer.

Blocking vs Non-Blocking

Most of the MPI point-to-point routines can be used in either blocking or non-blocking mode.

Blocking:

- A blocking send routine will only “return” after it is safe to modify the application buffer (your send data) for reuse.
- Safe means that modifications will not affect the data intended for the receive task. Safe does not imply that the data was actually received - it may very well be sitting in a system buffer.
- A blocking send can be synchronous which means there is handshaking occurring with the receive task to confirm a safe send.
- A blocking send can be asynchronous if a system buffer is used to hold the data for eventual delivery to the receive.
- A blocking receive only “returns” after the data has arrived and is ready for use by the program.

Blocking vs Non-Blocking

Non-blocking:

- Non-blocking send and receive routines behave similarly - they will return almost immediately. They do not wait for any communication events to complete, such as message copying from user memory to system buffer space or the actual arrival of message.
- Non-blocking operations simply “request” the MPI library to perform the operation when it is able. The user can not predict when that will happen.
- It is unsafe to modify the application buffer (your variable space) until you know for a fact the requested non-blocking operation was actually performed by the library. There are “wait” routines used to do this.
- Non-blocking communications are primarily used to overlap computation with communication and exploit possible performance gains.

Order and Fairness

MPI guarantees that messages will not overtake each other.

If a sender sends two messages (Message 1 and Message 2) in succession to the same destination, and both match the same receive, the receive operation will receive Message 1 before Message 2.

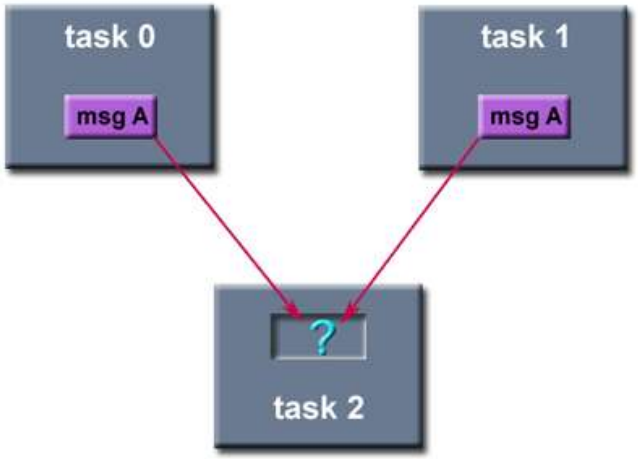
If a receiver posts two receives (Receive 1 and Receive 2), in succession, and both are looking for the same message, Receive 1 will receive the message before Receive 2.

Order rules do not apply if there are multiple threads participating in the communication operations.

Order and Fairness

MPI does not guarantee fairness - it's up to the programmer to prevent "operation starvation".

Example: task 0 sends a message to task 2. However, task 1 sends a competing message that matches task 2's receive. Only one of the sends will complete.



Non-blocking Message Passing Routines

Apart from the arguments in the send/receive function, the non-blocking functions (MPI_ISEND/MPI_IRECV) have an additional communication request handle for handling the pending message status.

- For sending, the program should not modify the application buffer until subsequent calls to MPI_Wait or MPI_Test indicate that the non-blocking send has completed.
- For receiving, the program must use calls to MPI_Wait or MPI_Test to determine when the non-blocking receive operation completes and the requested message is available in the application buffer.

In addition, there is also a Non-blocking synchronous send.

- It is similar to MPI_Isend(), except MPI_Wait() or MPI_Test() indicates when the destination process has received the message.

MPI Test

MPI_Test checks the status of a specified non-blocking send or receive operation.

- Checks if a single non-blocking operation (like MPI_Isend or MPI_Irecv) is complete.
- The “flag” parameter is returned logical true (1) if the operation has completed, and logical false (0) if not.
- For multiple non-blocking operations, the programmer can specify any, all or some completions.

MPI Test

MPI_Testany

- Checks multiple requests (from an array of non-blocking operations).
- Returns as soon as any one operation is complete.
- Gives the index of the completed request.

MPI_Testall

- Checks if **all** non-blocking operations in a request array are complete.
- Only return true when all the operations are finished

MPI_Testsome

- Checks multiple requests and returns the subset that has completed.
- Useful when some but not all operations might be done.

MPI Probe

Performs a non-blocking test for a message. The “wildcards” MPI_ANY_SOURCE and MPI_ANY_TAG may be used to test for a message from any source or with any tag.

The integer “flag” parameter is returned logical true (1) if a message has arrived, and logical false (0) if not.

For the C routine, the actual source and tag will be returned in the status structure as status.MPI_SOURCE and status.MPI_TAG.

Collective Communication Routines

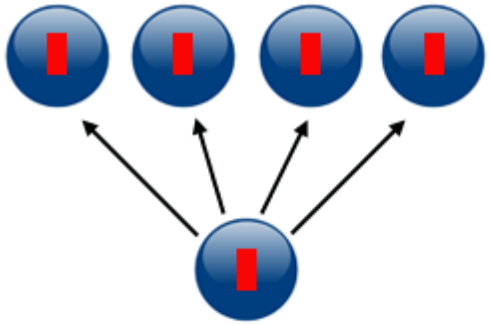
There are three types of collective communication routines available in MPI

Synchronization - processes wait until all members of the group have reached the synchronization point.

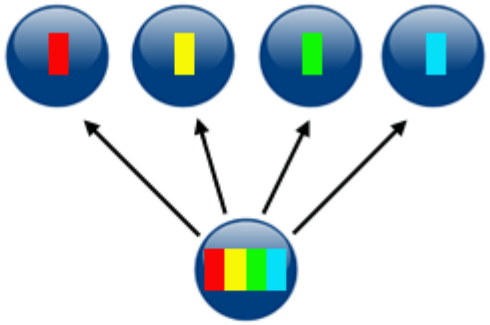
Data Movement - broadcast, scatter/gather, all to all.

Collective Computation (reductions) - one member of the group collects data from the other members and performs an operation (min, max, add, multiply, etc.) on that data.

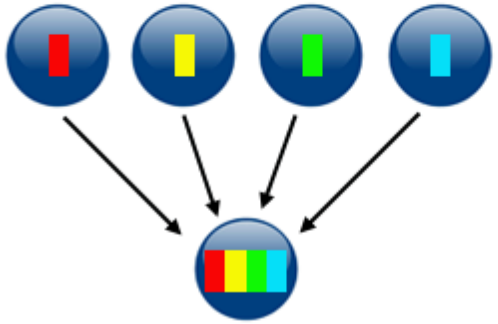
Collective Communication



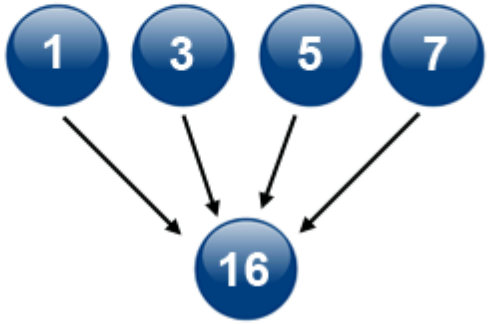
broadcast



scatter



gather



reduction

Collective Communication

Collective communication routines must involve all processes within the scope of a communicator. All processes are by default, members in the communicator MPI_COMM_WORLD.

Unexpected behavior, including program failure, can occur if even one task in the communicator doesn’t participate.

It is the programmer’s responsibility to ensure that all processes within a communicator participate in any collective operations.

Collective Communication

Collective communication routines do not take message tag arguments.

Collective operations within subsets of processes are accomplished by first partitioning the subsets into new groups and then attaching the new groups to new communicators

Can only be used with MPI predefined datatypes

With MPI-3, collective operations can be blocking or non-blocking. Only blocking operations are covered in this tutorial.

MPI Barrier

Synchronization operation. Creates a barrier synchronization in a group.

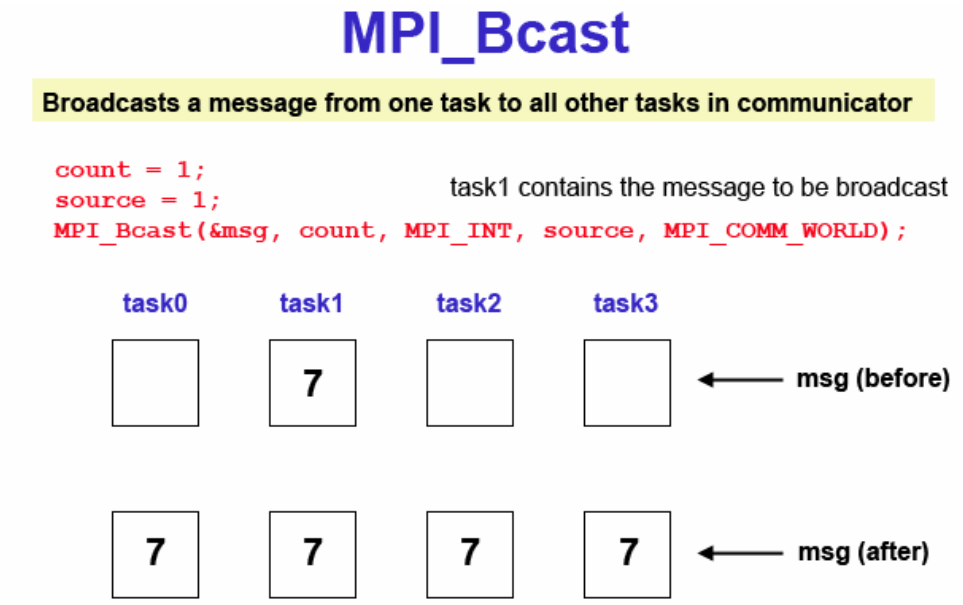
Each task, when reaching the MPI_Barrier call, blocks until all tasks in the group reach the same MPI_Barrier call.

- Then all tasks are free to proceed.

Syntax:
MPI_Barrier(comm);

MPI BCast

Data movement operation. Broadcasts (sends) a message from the process with rank “root” to all other processes in the group.

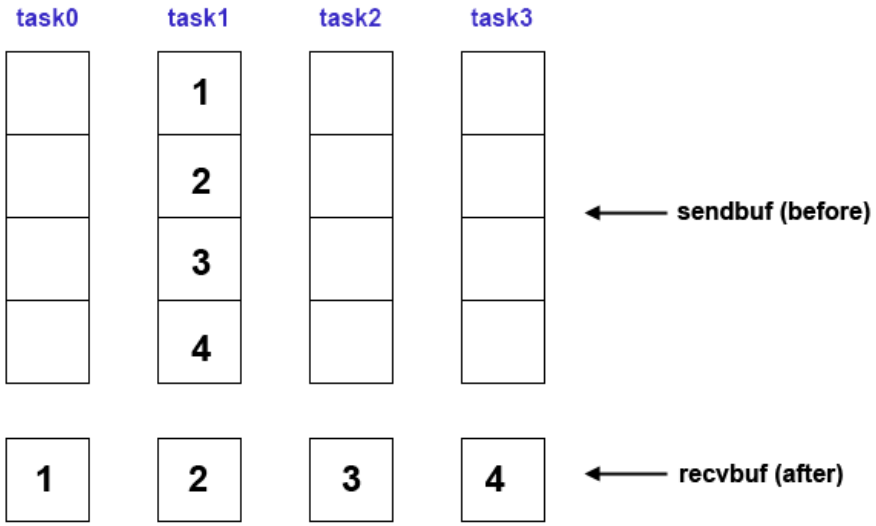


MPI Scatter

Data movement operation. Distributes distinct messages from a single source task to each task in the group.

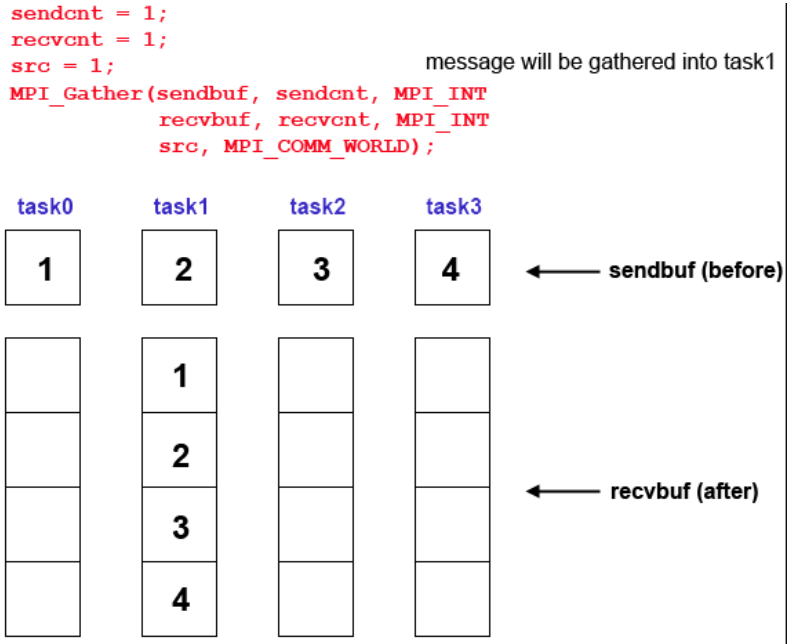
```
sendcnt = 1;
recvcnt = 1;
src = 1;
MPI_Scatter(sendbuf, sendcnt, MPI_INT
            recvbuf, recvcnt, MPI_INT
            src, MPI_COMM_WORLD);
```

task1 contains the data to be scattered



MPI Gather

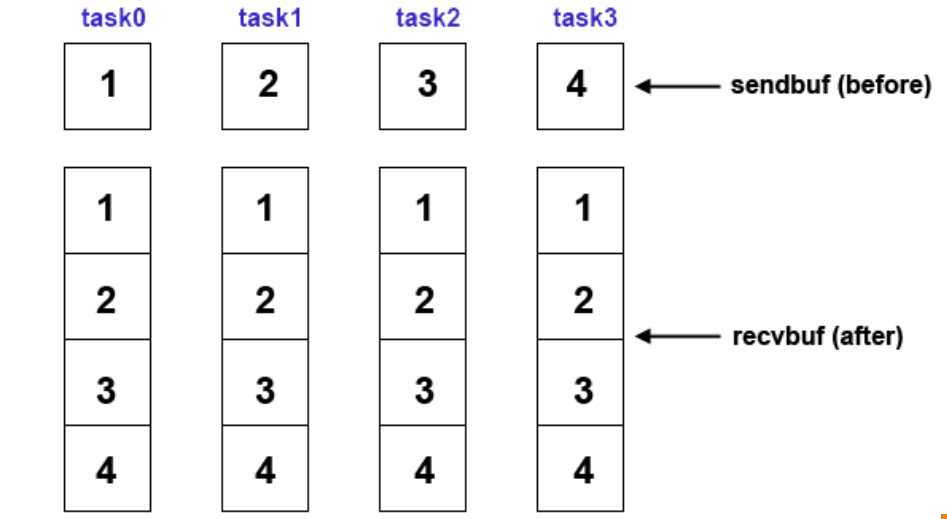
Data movement operation. Gathers distinct messages from each task in the group to a single destination task. This routine is the reverse operation of MPI_Scatter.



MPI Allgather

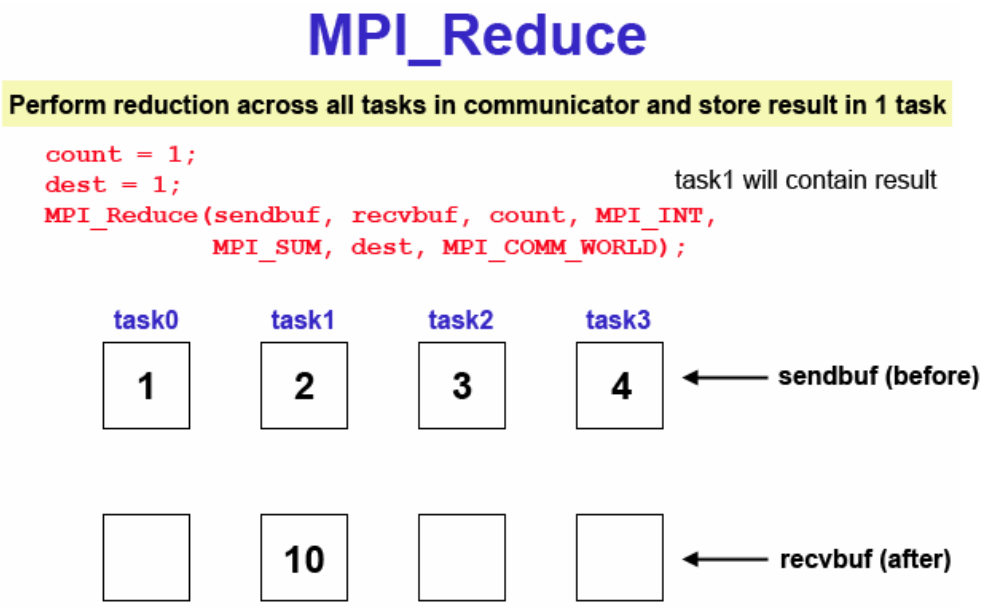
Concatenation of data to all tasks in a group. Each task in the group, in effect, performs a one-to-all broadcasting operation within the group.

```
sendcnt = 1;
recvcnt = 1;
MPI_Allgather(sendbuf, sendcnt, MPI_INT
              recvbuf, recvcnt, MPI_INT
              MPI_COMM_WORLD);
```



MPI Reduce

Collective computation operation. Applies a reduction operation on all tasks in the group and places the result in one task.



MPI Reduce

MPI Reduction Operation		C Data Types
MPI_MAX	maximum	integer, float
MPI_MIN	minimum	integer, float
MPI_SUM	sum	integer, float
MPI_PROD	product	integer, float
MPI_LAND	logical AND	integer
MPI_BAND	bit-wise AND	integer, MPI_BYTE
MPI_LOR	logical OR	integer
MPI_BOR	bit-wise OR	integer, MPI_BYTE
MPI_LXOR	logical XOR	integer
MPI_BXOR	bit-wise XOR	integer, MPI_BYTE
MPI_MAXLOC	max value and location	float, double and long double
MPI_MINLOC	min value and location	float, double and long double

Note from the MPI_Reduce man page

The operation is always assumed to be associative.

All predefined operations are also assumed to be commutative. Users may define operations that are assumed to be associative, but not commutative.

The “canonical” evaluation order of a reduction is determined by the ranks of the processes in the group.

- However, the implementation can take advantage of associativity, or associativity and commutativity in order to change the order of evaluation.
- This may change the result of the reduction for operations that are not strictly associative and commutative, such as floating=point addition.

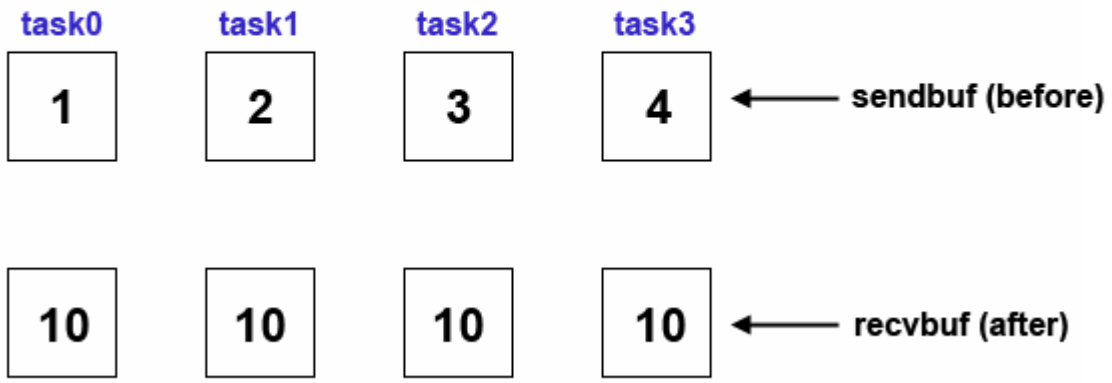
It is strongly recommended that MPI_REDUCE be implemented so that the same result be obtained whenever the function is applied on the same arguments, appearing in the same order.

- Note that this may prevent optimizations that take advantage of the physical location of processors.

MPI AllReduce

Collective computation operation + data movement. Applies a reduction operation and places the result in all tasks in the group. This is equivalent to an MPI_Reduce followed by an MPI_Bcast.

```
count = 1;
MPI_Allreduce(sendbuf, recvbuf, count, MPI_INT,
              MPI_SUM, MPI_COMM_WORLD);
```



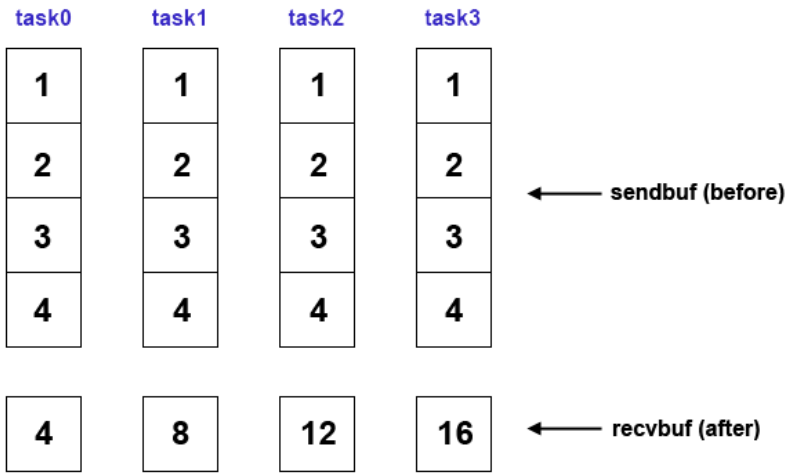
MPI Reduce Scatter

Collective computation operation + data movement. First does an element-wise reduction on a vector across all tasks in the group. Next, the result vector is split into disjoint segments and distributed across the tasks. This is equivalent to an MPI_Reduce followed by an MPI_Scatter operation.

MPI_Reduce_scatter

Perform reduction on vector elements and distribute segments of result vector across all tasks in communicator

```
recvcnt = 1;
MPI_Reduce_scatter(sendbuf, recvbuf, recvcount,
                   MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



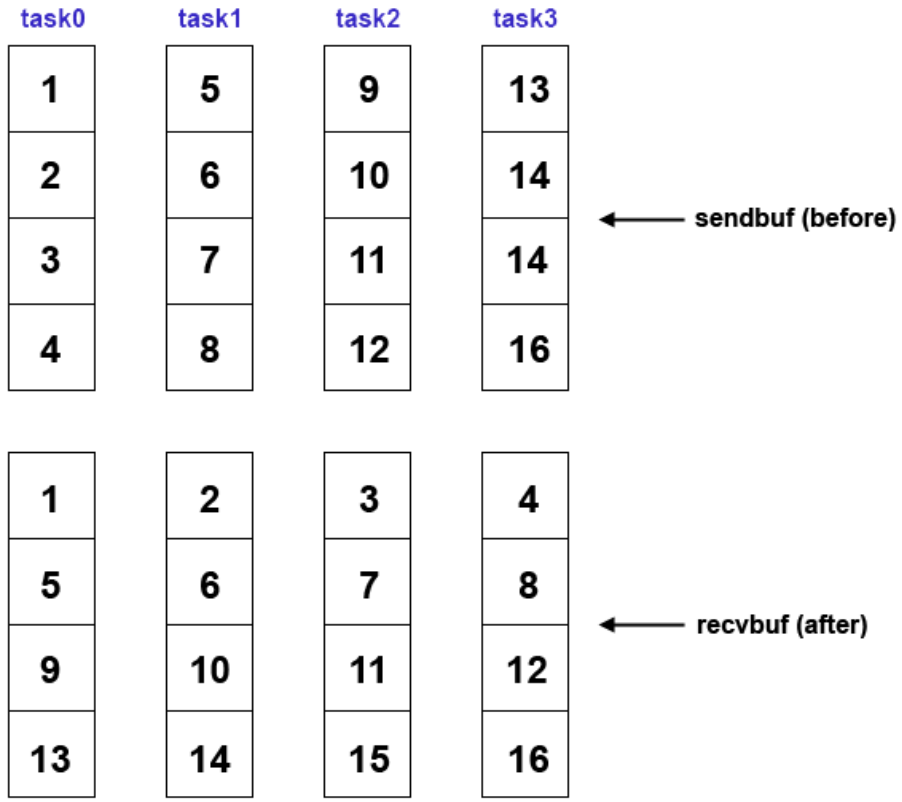
MPI All to All

Data movement operation. Each task in a group performs a scatter operation, sending a distinct message to all the tasks in the group in order by index.

MPI_Alltoall

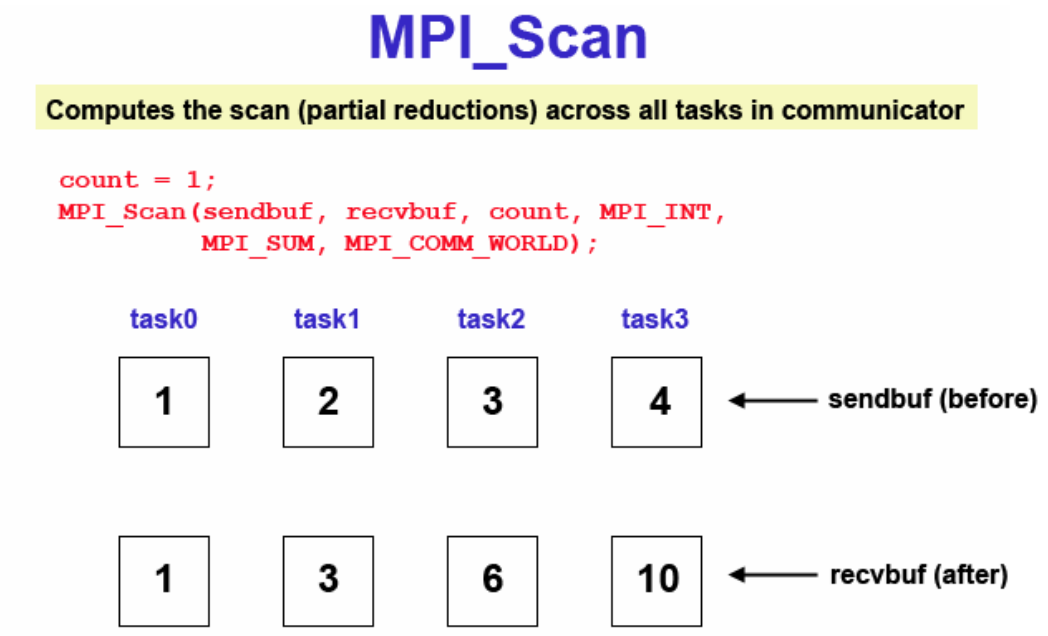
Scatter data from all tasks to all tasks in communicator

```
sendcnt = 1;
recvcnt = 1;
MPI_Alltoall(sendbuf, sendcnt, MPI_INT
             recvbuf, recvcnt, MPI_INT
             MPI_COMM_WORLD);
```



MPI Scan

Performs a scan operation with respect to a reduction operation across a task group.



Derived Data Type

MPI also provides facilities for you to define your own data structures based upon sequences of the MPI primitive data types.

- Such user defined structures are called derived data types.

Primitive data types are contiguous. Derived data types allow you to specify non-contiguous data in a convenient manner and to treat it as though it was contiguous.

Derived Data Type

MPI provides several methods for constructing derived data types:

Contiguous

Vector

Indexed

Struct

Derived Data Types Routines

MPI Type contiguous

MPI_Type_contiguous (count,oldtype,&newtype)

- Produces a new data type by making count copies of an existing data type.

MPI_Type_vector

MPI_Type_vector (count,blocklength,stride,oldtype,&newtype)

- Like contiguous but allows for regular gaps (stride) in the displacements.
- MPI_Type_hvector is identical to MPI_Type_vector except that stride is specified in bytes.

Derived Data Types Routines

MPI_Type_indexed

MPI_Type_indexed (count,blocklens[],offsets[],old_type,&newtype)

- An array of displacements of the input data type is provided as the map for the new data type.
- MPI_Type_hindexed is identical to MPI_Type_indexed except that offsets are specified in bytes.

MPI_Type_struct

MPI_Type_struct (count,blocklens[],offsets[],old_types,&newtype)

- The new data type is formed according to completely defined map of the component data types.

MPI Type Contiguous

MPI_ Type_contiguous

```
count = 4;
MPI_Type_contiguous(count, MPI_FLOAT, &rowtype);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

a[4][4]

```
MPI_Send(&a[2][0], 1, rowtype, dest, tag, comm);
```

9.0	10.0	11.0	12.0

1 element of rowtype

MPI Type Vector

MPI_ Type_vector

count = 4; blocklength = 1; stride = 4;
MPI_Type_vector(count, blocklength, stride, MPI_FLOAT, &columntype);

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

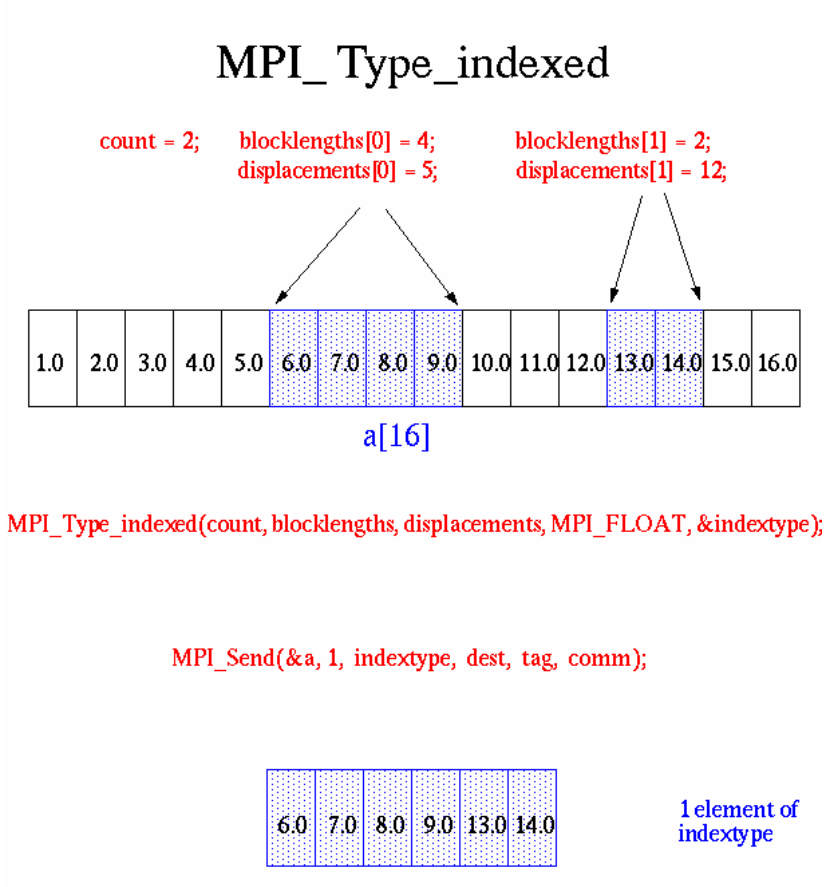
a[4][4]

MPI_Send(&a[0][1], 1, columntype, dest, tag, comm);

2.0	6.0	10.0	14.0

1 element of columntype

MPI Type Indexed



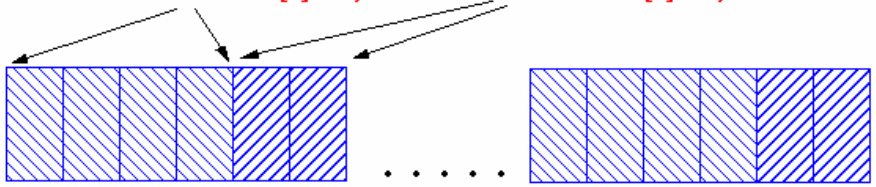
MPI Type Struct

MPI_ Type_struct

```
typedef struct { float x, y, z, velocity; int n, type; } Particle;
Particle particles[NELEM];

MPI_Type_extent(MPI_FLOAT, &extent);

count = 2; oldtypes[0] = MPI_FLOAT;   oldtypes[1] = MPI_INT
           offsets[0] = 0;            offsets[1] = 4 * extent;
           blockcounts[0] = 4;        blockcounts[1] = 2;
```



particles[NELEM]

```
MPI_Type_struct(count, blockcounts, offsets, oldtypes, &particletype);

MPI_Send(particles, NELEM, particletype, dest, tag, comm);
```

Sends entire (NELEM) array of particles, each particle being comprised four floats and two integers.

Additional functions

MPI_Type_extent

- Returns the size in bytes of the specified data type. Useful for the MPI subroutines that require specification of offsets in bytes.

MPI_Type_commit

- Commits new datatype to the system. Required for all user constructed (derived) datatypes.

MPI_Type_free

- Deallocates the specified datatype object. Use of this routine is especially important to prevent memory exhaustion if many datatype objects are created, as in a loop.

Groups

A group is an ordered set of processes.

Each process in a group is associated with a unique integer rank.

- Rank values start at zero and go to N-1, where N is the number of processes in the group.

In MPI, a group is represented within system memory as an object.

- It is accessible to the programmer only by a “handle”.

A group is always associated with a communicator object.

Communicator

A communicator encompasses a group of processes that may communicate with each other.

- All MPI messages must specify a communicator.

In the simplest sense, the communicator is an extra “tag” that must be included with MPI calls.

Like groups, communicators are represented within system memory as objects and are accessible to the programmer only by “handles”.

- For example, the handle for the communicator that comprises all tasks is MPI_COMM_WORLD.

From the programmer’s perspective, a group and a communicator are one.

- The group routines are primarily used to specify which processes should be used to construct a communicator.

Primary Purposes of Group and Communicator Objects

Allow you to organize tasks, based upon function, into task groups.

Enable Collective Communications operations across a subset of related tasks.

Provide basis for implementing user defined virtual topologies.

Provide for safe communications.

Programming Considerations and Restrictions

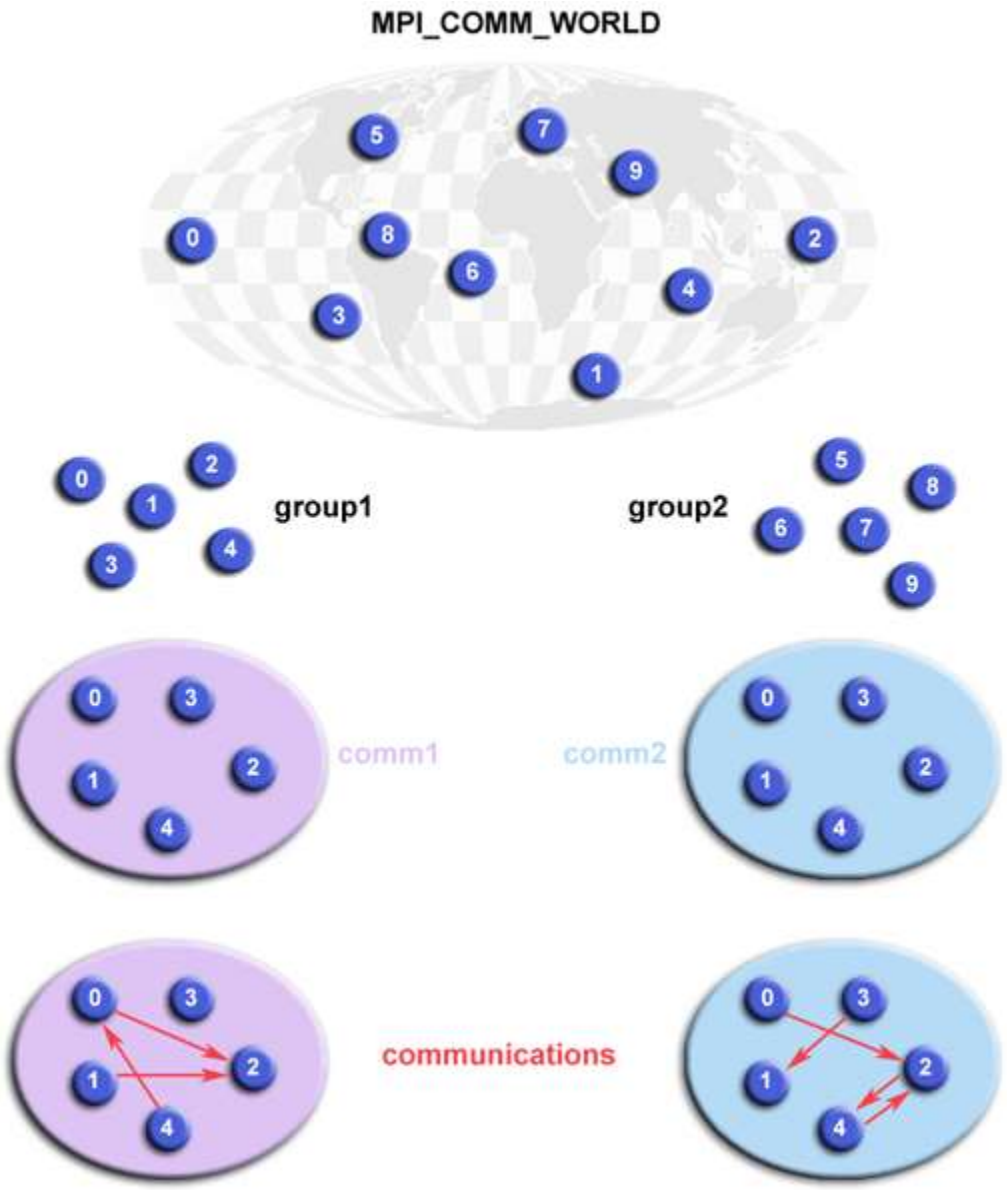
Groups/communicators are dynamic - they can be created and destroyed during program execution.

Processes may be in more than one group/communicator. They will have a unique rank within each group/communicator.

MPI provides over 40 routines related to groups, communicators, and virtual topologies.

Example

1. Extract handle of global group from MPI_COMM_WORLD using MPI_Comm_group
2. Form new group as a subset of global group using MPI_Group_incl
3. Create new communicator for new group using MPI_Comm_create
4. Determine new rank in new communicator using MPI_Comm_rank
5. Conduct communications using any MPI message passing routine
6. When finished, free up new communicator and group (optional) using MPI_Comm_free and MPI_Group_free


MPI_COMM_WORLD
group1
group2
comm1
comm2
communications

Virtual topologies

In terms of MPI, a virtual topology describes a mapping/ordering of MPI processes into a geometric “shape”.

The two main types of topologies supported by MPI ar

- Cartesian (grid)
- Graph

MPI topologies are virtual - there may be no relation between the physical structure of the parallel machine and the process topology.

Virtual topologies are built upon MPI communicators and groups.

They must be “programmed” by the application developer.

Virtual Topologies

Why Use Them?

Convenience

- Virtual topologies may be useful for applications with specific communication patterns - patterns that match an MPI topology structure.
- For example, a Cartesian topology might prove convenient for an application that requires 4-way nearest neighbor communications for grid based data.

Communication Efficiency

- Some hardware architectures may impose penalties for communications between successively distant “nodes”.
- A particular implementation may optimize process mapping based upon the physical characteristics of a given parallel machine.
- The mapping of processes into an MPI virtual topology is dependent upon the MPI implementation, and may be totally ignored.

Example

A simplified mapping of processes into a Cartesian virtual topology appears below:

0 (0,0)	1 (0,1)	2 (0,2)	3 (0,3)
4 (1,0)	5 (1,1)	6 (1,2)	7 (1,3)
8 (2,0)	9 (2,1)	10 (2,2)	11 (2,3)
12 (3,0)	13 (3,1)	14 (3,2)	15 (3,3)